

INFORME DE PRUEBAS DE SOFTWARE

Pruebas unitarias y de integración

Parkings Together

Marketplace P2P de estacionamientos

Documento	Informe de pruebas de software
Proyecto	Parkings Together — Marketplace P2P de estacionamientos
Versión	1.0
Fecha	12 de junio de 2026
Asignatura	
Docente	
Integrantes	
Resultado global	10 / 10 pruebas aprobadas (100 %)

Tabla de contenido

1. Introducción y objetivo	2
2. Alcance y entorno de pruebas	2
3. Estrategia y tipos de prueba	2
4. Resumen de resultados	3
5. Detalle por suite de pruebas	3
5.1. api.test.js — Contratos del BFF (node:test)	3
5.2. reserveService.test.js — Saga de reservas (Jest)	4
5.3. api.timeout.test.js — Resiliencia del BFF (Jest)	4
6. Cobertura de código	4
7. Evidencia de ejecución	5
8. Cómo reproducir las pruebas	5
9. Alcance, limitaciones y mejoras futuras	6
10. Conclusiones	6

Nota: en Microsoft Word, haga clic derecho sobre el índice y seleccione «Actualizar campos» para numerar las páginas.

1. Introducción y objetivo

El presente informe documenta las pruebas automatizadas implementadas para el proyecto Parkings Together, una plataforma tipo marketplace de intermediación (P2P) para el arriendo de estacionamientos entre particulares. El objetivo es verificar, mediante pruebas reproducibles, que la lógica de negocio crítica y los contratos de las interfaces (API) del sistema se comportan según lo esperado y se mantienen estables ante cambios.

Las pruebas se ejecutan de forma aislada, sin depender de servicios externos: tanto la base de datos como la red se sustituyen por dobles de prueba (mocks). De este modo, las pruebas son rápidas, deterministas y reproducibles en cualquier entorno, lo que las hace aptas como evidencia objetiva de la calidad del software.

2. Alcance y entorno de pruebas

El alcance de esta batería de pruebas cubre la lógica de dominio del microservicio de reservas (patrón Saga con compensación y control de capacidad bajo CQRS) y el contrato del orquestador de servicios (BFF) que el frontend consume mediante rutas de mismo origen. La siguiente tabla resume el entorno técnico utilizado.

Aspecto	Detalle
Lenguaje / runtime	JavaScript · Node.js (v24; requiere ≥ 22)
Pruebas de integración (BFF)	node:test — runner de pruebas nativo de Node.js
Pruebas unitarias y de dominio	Jest (babel-jest para módulos ESM)
Cobertura de código	Istanbul (reporte de texto y HTML lcov-report)
Aislamiento	Mock de fetch global y mock de repositorio (jest.mock): sin base de datos ni red reales
Ubicación de las pruebas	apps/web/tests/ y apps/ms-reservas/tests/

3. Estrategia y tipos de prueba

Se aplicó una estrategia mixta que combina pruebas unitarias de dominio con pruebas de integración del contrato de las interfaces. Cada tipo de prueba valida una preocupación distinta del sistema:

Tipo	Qué valida	Suite
Integración de API (BFF)	Contrato de las rutas del orquestador: estructura de respuesta y manejo de éxito y error	api.test.js
Dominio / Saga	Reserva atómica con compensación (rollback) y control de capacidad (CQRS)	reserveService.test.js
Resiliencia	Timeout estricto y respuesta de respaldo (fallback) del BFF ante latencia del backend	api.timeout.test.js

4. Resumen de resultados

Se ejecutaron 10 pruebas automatizadas distribuidas en 3 suites. La totalidad de las pruebas resultó aprobada, en un tiempo de ejecución inferior a un segundo y sin dependencias externas.

Suite	Framework	Casos	Resultado
api.test.js	node:test	6	6 / 6 aprobadas
api.timeout.test.js	Jest	2	2 / 2 aprobadas
reserveService.test.js	Jest	2	2 / 2 aprobadas
TOTAL	—	10	10 / 10 (100 %)

5. Detalle por suite de pruebas

5.1. api.test.js — Contratos del BFF (node:test)

Prueba la estructura de respuesta de las rutas de mismo origen `/api/mapas/search` y `/api/reservas/reserve` utilizando un mock de fetch, sin base de datos real. Verifica tanto los caminos de éxito como el manejo de errores del orquestador (BFF). Casos cubiertos:

#	Caso	Resultado
1	Mapas: actualiza la ocupación correctamente	Aprobado
2	Mapas: rechaza la actualización al exceder la capacidad (PATCH)	Aprobado
3	Reservas: verifica disponibilidad cuando hay plaza libre	Aprobado
4	Reservas: verifica disponibilidad cuando la plaza está llena	Aprobado

5	Reservas: crea la reserva con éxito	Aprobado
6	Reservas: maneja el error cuando la Saga falla	Aprobado

5.2. reserveService.test.js — Saga de reservas (Jest)

Módulo: `apps/ms-reservas/app/api/v1/reserve/services/reserva.service.js`.

Demuestra la robustez transaccional del dominio de reservas mediante el patrón Saga con compensación:

- **Control de capacidad (CQRS):** rechaza la reserva si el estacionamiento ya está lleno y, en ese caso, no llega a crear la reserva.
- **Compensación (rollback):** si la operación de aumento de ocupación falla (simulando un error de concurrencia), la Saga revierte la reserva ya creada, garantizando la integridad de los datos («Saga Compensada»).

5.3. api.timeout.test.js — Resiliencia del BFF (Jest)

Verifica que el cliente del BFF devuelva los datos cuando la respuesta llega a tiempo, y que aborte la petición mediante `AbortController` entregando una respuesta de respaldo segura (`{ success: false, error: true, fallback: true, data: [] }`) cuando la petición supera el límite estricto de 8 segundos. De este modo se demuestra el manejo controlado de la latencia o indisponibilidad del backend.

6. Cobertura de código

La siguiente tabla presenta la cobertura de los módulos ejercitados por las suites Jest, generada con Istanbul. Cabe destacar que las ramas (branches) del servicio de reservas —donde reside la lógica crítica de la Saga— se encuentran cubiertas al 100 %.

Archivo	% Stmts	% Branch	% Funcs	% Lines
All files	62.71	47.36	30.76	62.50
<code>reserva.service.js</code>	78.57	100	50	78.57
<code>api.js</code>	60	40	33.33	59.25

La cobertura se concentra deliberadamente en la lógica de dominio y en el cliente del orquestador. El archivo `api.test.js` (6 pruebas del BFF) se ejecuta con el runner nativo `node:test`, por lo que su aporte de cobertura no aparece en el reporte de Jest, aunque su contrato sí queda verificado.

7. Evidencia de ejecución

A continuación se transcribe la salida real de la ejecución de las pruebas, de modo que el informe es autocontenido y verificable. Las capturas pueden regenerarse ejecutando los comandos de la sección 8.

Evidencia 1 — Pruebas de integración del BFF (node:test):

```
PS apps/web> node --test tests/api.test.js
✓ BFF - Mapas: Deberia actualizar ocupacion correctamente
✓ BFF - Mapas: Deberia fallar al exceder capacidad en PATCH
✓ BFF - Reservas (Saga): Deberia verificar disponibilidad
✓ BFF - Reservas (Saga): Deberia rechazar si esta lleno
✓ BFF - Reservas (Saga): Deberia crear reserva con exito
✓ BFF - Reservas (Saga): Deberia manejar el error si la Saga falla
i tests 6 | i pass 6 | i fail 0
```

Evidencia 1: 6/6 pruebas del BFF aprobadas (node:test).

Evidencia 2 — Pruebas Jest con cobertura (dominio + resiliencia):

```
PS (raiz)> npx jest --coverage
PASS apps/ms-reservas/tests/reserveService.test.js
PASS apps/web/tests/api.timeout.test.js
-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines |
-----|-----|-----|-----|
All files |    62.71 |    47.36 |    30.76 |    62.50 |
  reserva.service.js |    78.57 |    100   |     50   |    78.57 |
  api.js   |     60   |     40   |    33.33 |    59.25 |
-----|-----|-----|-----|
Test Suites: 2 passed, 2 total
Tests:      4 passed, 4 total
```

Evidencia 2: 2 suites y 4 pruebas Jest aprobadas, con reporte de cobertura.

8. Cómo reproducir las pruebas

Desde la raíz del monorepo, tras instalar las dependencias, las pruebas se reproducen con los siguientes comandos:

```
# Instalar dependencias (una sola vez)
npm install

# 1) Pruebas de integracion del BFF (node:test) -> 6 pruebas
cd apps/web
npm test          # ejecuta: node --test tests/api.test.js

# 2) Pruebas de dominio y resiliencia (Jest) + cobertura -> 4 pruebas
cd ../..
npx jest --coverage # genera coverage/lcov-report/index.html
```

El reporte HTML de cobertura queda disponible en `coverage/lcov-report/index.html` para su revisión en el navegador.

9. Alcance, limitaciones y mejoras futuras

En aras de la transparencia, se documentan los límites actuales de la batería de pruebas y las líneas de trabajo recomendadas:

- La cobertura está concentrada en la lógica de dominio (Saga de reservas) y en el cliente del BFF; los controladores de ruta de Next.js y el paquete de acceso a datos (packages/supabase-db) aún no cuentan con pruebas dedicadas.
- La cobertura global de funciones es perfectible: se recomienda añadir pruebas a más controladores de ruta del BFF para elevarla por encima del 60 % en todos los componentes.
- Se recomienda incorporar pruebas de extremo a extremo (end-to-end) sobre el flujo de reserva completo, una vez disponible un entorno de integración con base de datos de prueba.

10. Conclusiones

Las 10 pruebas automatizadas implementadas se ejecutan de forma satisfactoria (100 % aprobadas), de manera determinista y sin dependencias externas, validando los componentes centrales del sistema.

La lógica crítica de negocio queda respaldada: el patrón Saga del microservicio de reservas valida el control de capacidad (CQRS) y revierte correctamente ante fallos (compensación, con sus ramas cubiertas al 100 %), mientras que el orquestador (BFF) gestiona de forma controlada los escenarios de error y de latencia mediante timeout y respuesta de respaldo.

En conjunto, la batería de pruebas constituye evidencia objetiva de que los componentes centrales de Parkings Together se comportan según lo esperado y quedan protegidos ante regresiones, sentando además una base clara para ampliar la cobertura a las áreas señaladas en la sección 9.